GHC.Generics: https://hackage.haskell.org/package/base/docs/GHC-Generics.html

```
-- GHC.Generics

-- type meta
newtype M1 (i :: Type) (c :: Meta) (f :: k -> Type)
(p :: k) =            -- M = metadata
  M1 { unM1 :: f p }
                 -- e.g. type constructor, data
constructor, field
newtype Rec1 (f :: k -> Type) (p :: k) =
                 -- Recursive type
  Rec1 { Rec1 :: f p }
newtype K1 (i :: Type) c (p :: k) =
                 -- K = constant value of type c
  K1 { unK1 :: c }
                 -- e.g. Just (x :: c)
data U1 (p :: k) = U1
                 -- nullary data constructor e.g. True
data V1 (p :: k)
                 -- V = void. type with no constructor
i.e. Void
newtype Par1 p =
                 -- Par = Parameter
  Par1 { uPar1 :: p }

data D
data C
data S
type D1 = M1
D
    -- type constructor
type C1 = M1
C
    -- data constructor
type S1 = M1
S
    -- data constructor value
data R
type Rec0 = K1 R

-- Operators
data (f :: k -> Type) :+: (g :: k -> Type) = L1 (f p) |
R1 (g p)        -- algebraic sum e.g. True | False
data (f :: k -> Type) :*: (g :: k -> Type) = (f p) :*:
(g p)            -- product e.g. A a b
newtype ((f :: k2 -> Type) :.: (g :: k1 -> k2)) (p ::
k1) =
  Comp { unComp :: f (g p) }


-- Metadata
data Meta
```

```
 = MetaData Symbol Symbol Symbol Bool
                            -- type constructor metadata
 | MetaCons Symbol Fixity Bool
                           -- data constructor metadata
 | MetaSel (Maybe Symbol) SourceUnpackedness
SourceStrictness DecidedStrictness    -- constr

data Fixity
 = Prefix
 | Infix Associativity Int
data FixityI
 = PrefixI
 | InfixI AssociativityI Nat
data Associativity
 = LeftAssociative
 | RightAssociative
 | NotAssociative
data SourceUnpackedness
 = SourceNoUnpack
 | SourceUnpack
data SourceStrictness
 = NoSourceStrictness
 | SourceLazy
 | SourceStrict
data DecidedStrictness
 = DecidedLazy
 | DecidedStrict
 | DecidedUnpack


-- Reflection
class Generic a where
 type Rep a :: Type -> Type
 to :: Rep a x -> a
 from :: a -> Rep a x

class Generic1 (f :: k -> Type) where
 type Rep1 f :: k -> Type
 to1 :: f a -> Rep1 f a
 from1 :: Rep1 f a -> f a

class Datatype d where
  datatypeName :: t d (f :: k -> Type) (a :: k) ->
String
  moduleName :: t d (f :: k -> Type) (a :: k) -> String
  packageName :: t d (f :: k -> Type) (a :: k) -> String
  isNewtype :: t d (f :: k -> Type) (a :: k) -> Bool
instance (KnownSymbol n, KnownSymbol m, KnownSymbol p,
SingI nt) => Datatype ('MetaData n m p nt :: Meta)

class Constructor c where
  conName :: t c (f :: k -> Type) (a :: k) -> String
  conFixity :: t c (f :: k -> Type) (a :: k) -> Fixity
  conIsRecord :: t c (f :: k -> Type) (a :: k) -> Bool
```

```
instance (KnownSymbol n, SingI f, SingI r) => Datatype
('MetaCons n f r :: Meta)

class Selector s where
  selName :: t s (f :: k -> Type) (a :: k) -> String
  selSourceUnpackedness :: t s (f :: k -> Type) (a :: k)
-> SourceUnpackedness
  selSourceStrictness :: t s (f :: k -> Type) (a :: k)
-> SourceStrictness
  selDecidedStrictness :: t s (f :: k -> Type) (a :: k)
-> DecidedStrictness
instance (SingI mn, SingI su, SingI ss, SingI ds) =>
Datatype ('MetaSel n su ss ds  :: Meta)
```

Generics
Reflection system i.e. given a **value**, get info about its **type**
e.g. get name of type, name of fields
Generics provides an **isomoprhic representation** (i.e. 1-1) of a value with extra **metadata** for reflection in the type
e.g.

```
-- Generics.Rep Bool
Rep Bool = M1 D ('MetaData "Bool" "GHC.Types" "ghc-prim"
'False) (       -- meta form of type=Bool
    M1 C ('MetaCons "False" 'PrefixI 'False) U1
    :+:
    M1 C ('MetaCons "True" 'PrefixI 'False) U1
  ) x

-- meta representation
b0 :: Rep Bool                    -- Rep
Bool: meta form of type=Bool
b1 :: Rep Bool
b0 = M1 (R1 (M1 U1))              -- b0: meta form of
value=True
b1 = M1 (L1 (M1 U1))              -- b1: meta form of
value=False

-- isomoprhism
from :: Bool -> Rep Bool          -- value -> meta form
to :: Rep Bool -> Bool            -- meta form -> value
from True == b0
to b0 == True
from False == b1
to b1 == False
```

Main types used in the representation

type **M1 D** ('MetaData "<type name>" …)	type constructor
type **M1 C** ('MetaCons "<constructor name>" …)	data constructor
type **M1 S** ('MetaSel (Just "<field name>" …)	named field
e.g. MyData { value = 1 }	

type **M1 S** ('MetaSel Nothing …) data constructor
value e.g. Just 1
type **U1** nullary data
constructor e.g. True
type **K1 R** c value of type c
(R=recursive) e.g. 1 :: Int with c=Int
(M1 C …) **:+:** (M1 C …) algebraic sum
e.g. True | False
(M1 S …) **:*:** (M1 S …) algebraic
product e.g. (a, b)

Example: get type constructor info
Use **Datatype** class

```
{-# LANGUAGE DerivingStrategies #-}

import GHC.Generics

data MyData = MyData {name :: String, value :: Int}
  deriving stock (Show, Eq, Generic)

data Alpha = A | B | C | D
  deriving stock (Show, Eq, Generic)

-- e.g.
datatypeName (from A)                        -- "Alpha"
datatypeName (from (MyData "a" 1))           -- "MyData"
```

Example: get data constructor info
Use type classes + **Constructor** class

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE TypeOperators #-}

import GHC.Generics

class ConstructorInfo m where
  getConstructorName :: m a -> String

instance ConstructorInfo a => ConstructorInfo (M1 D
(s :: Meta) a) where
  getConstructorName (M1 x) = getConstructorName x

instance Constructor s => ConstructorInfo (M1 C (s ::
Meta) a) where
  getConstructorName = conName

instance (ConstructorInfo f, ConstructorInfo g) =>
```

```
ConstructorInfo (f :+: g) where
  getConstructorName (L1 x) = getConstructorName x
  getConstructorName (R1 x) = getConstructorName x

data MyData = MyData {name :: String, value :: Int}
  deriving stock (Show, Eq, Generic)

data Alpha = A | B | C | D
  deriving stock (Show, Eq, Generic)

-- e.g.
getConstructorName (from A)                    -- "B"
getConstructorName (from B)                    -- "A"
getConstructorName (from $ MyData "a" 1)       --
"MyData"
```

Example: common types
Boolean

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import GHC.Generics

type BoolType :: forall a. a -> Type           -- Rep
Bool x
type BoolType x =
  M1 D ('MetaData "Bool" "GHC.Types" "ghc-prim" 'False)
(
    M1 C ('MetaCons "False" 'PrefixI 'False) U1
    :+:
    M1 C ('MetaCons "True" 'PrefixI 'False) U1
  ) x

-- e.g.
metaTrue :: BoolType x
-- metaTrue :: Rep Bool x              -- same
metaTrue = M1 (R1 (M1 U1))
metaFalse :: BoolType x
-- metaFalse :: Rep Bool x             -- same
metaFalse = M1 (L1 (M1 U1))
metaTrue === from True                 -- True
metaFalse === from False               -- True
```

Maybe a

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import GHC.Generics
```

```
type MaybeType :: forall a. Type -> a -> Type
             -- Rep (Maybe a) x
type MaybeType a x =
  M1 D ('MetaData "Maybe" "GHC.Maybe" "base" 'False) (
    M1 C ('MetaCons 'Nothing" 'PrefixI 'False) U1
    :+:
    M1 C ('MetaCons "Just" 'PrefixI 'False)
          (M1 S ('MetaSel 'Nothing 'NoSourceUnpackedness
'NoSourceStrictness 'DecidedLazy) (K1 R a))
  ) x

-- e.g.
metaJust1 :: MaybeType Int x
-- metaJust1 :: Rep (Maybe a) x                    --
same
metaJust1 = M1 (R1 (M1 (M1 (K1 1))))
metaNothing :: MaybeType a x
-- metaNothing :: Rep (Maybe a) x                  --
same
metaNothing = M1 (L1 (M1 U1))
metaJust1 == from (Just 1)
metaNothing @Int == from Nothing
```

Either a b

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import GHC.Generics

type EitherType :: forall a. Type -> Type -> a -> Type
             -- Rep (Either a b) x
type EitherType a b x =
  M1 D ('MetaData "Either" "Data.Either" "base" 'False)
(
    M1 C ('MetaCons "Left" 'PrefixI 'False)
      (M1 S ('MetaSel 'Nothing 'NoSourceUnpackedness
'NoSourceStrictness 'DecidedLazy) (K1 R a))
    :+:
    M1 C ('MetaCons "Right" 'PrefixI 'False)
      (M1 S ('MetaSel 'Nothing 'NoSourceUnpackedness
'NoSourceStrictness 'DecidedLazy) (K1 R b))
    ) x

-- e.g.
metaLeftString :: EitherType String a x
-- metaLeftString :: Rep (Either a String)
x            -- same
metaLeftString = M1 (L1 (M1 ( M1 (K1 "ERROR"))))
metaRight1 :: EitherType a Int x
-- metaRight1 :: Rep (Either a
Int                       -- same
```

```
metaRight1 = M1 (R1 (M1 ( M1 (K1 1))))
metaLeftString @Int == from (Left "ERROR")
    -- True
metaRight1 @String == from (Right 1)
    -- True
```

(a,b)

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
import GHC.Generics

type PairType a b x =
  M1 D ('MetaData "(,)" "GHC.Tuple" "ghc-prim" 'False) (
    M1 C ('MetaCons "(,)" 'PrefixI 'False) (
      M1 S ('MetaSel 'Nothing 'NoSourceUnpackedness
'NoSourceStrictness 'DecidedLazy) (K1 R a)
    :*:
      M1 S ('MetaSel 'Nothing 'NoSourceUnpackedness
'NoSourceStrictness 'DecidedLazy) (K1 R b)
                                          )
    ) x

-- e.g.
metaPair :: PairType Int String x
-- metaPair :: Rep (Int, String) x                 --
same
metaPair = M1 (M1 (M1 (K1 1) :*: M1 (K1 "A")))
metaPair == from (1, "A")                          --
True
```

e.g. user defined type with various fields

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
import GHC.Generics

data SomeThing a = MkSomeThing
  { v1 :: {-# UNPACK #-} !String,             --
unpacked + Strict
    v2 :: String,
    v3 :: a
  }
  deriving (Generic, Show, Eq)

type SomeThingType :: Type -> Type             -- Rep
(Something Int)
type SomeThingType x =
  M1 D ('MetaData "SomeThing" "MyModule" "myPackage"
'False)
    (M1 C ('MetaCons "MkSomeThing" 'PrefixI 'True)
        (M1 S ('MetaSel ('Just "v1") 'SourceUnpack
'SourceStrict 'DecidedStrict) (K1 R String)
```

```
        :*: M1 S ('MetaSel ('Just "v2")
'NoSourceUnpackedness 'NoSourceStrictness 'DecidedLazy)
(K1 R String)
        :*: M1 S ('MetaSel ('Just "v3")
'NoSourceUnpackedness 'NoSourceStrictness 'DecidedLazy)
(K1 R Int)
        )
    )
    x

-- e.g.
y :: SomeThingType x
-- y :: Rep (SomeThing Int) x              -- same
y = from (MkSomething "" "" 1)
```